

锁

自旋锁spinlock

kernel / spinlock.h

```
1 // Mutual exclusion lock.
2 struct spinlock {
3     uint locked;      // Is the lock held?
4
5     // For debugging:
6     char *name;        // Name of lock.
7     struct cpu *cpu;   // The cpu holding the lock.
8 };
9
```

- locked
 - 0:该锁可获得
 - 非0: 锁被持有
- 获得acquire锁

```
void acquire(struct spinlock *lk)
{
    for(;;){
        if(lk->locked==0){
            lk->locked=1;
            break;
        }
    }
}
```

保证原子性

- 可能两个进程同时设置该锁，违背互斥
- 使用库函数__sync_lock_test_and_set函数——本质是amoswap r,a指令（原子地执行序列，使用特殊的硬件来防止其他cpu使用读写之间的内存地址）

```
void
acquire(struct spinlock *lk)
{
    push_off();
    if(holding(lk))
        panic("acquire");

    while(__sync_lock_test_and_set(&lk->locked, 1) != 0)
        ;

    __sync_synchronize();
    lk->cpu = mycpu();
}
```

- 释放release锁

```

void
release(struct spinlock *lk)
{
    if(!holding(lk))
        panic("release");

    lk->cpu = 0;

    __sync_synchronize();
    __sync_lock_release(&lk->locked);

    pop_off();//启用中断
}

```

1. 自旋锁:

- 自旋锁是一种基于忙等待（busy-waiting）的同步机制，即当线程尝试获取锁时，如果锁已被其他线程持有，则该线程会一直处于忙等待状态，**不会放弃 CPU 的控制权**。
- 适用于——临界区很短且锁的争用不频繁的情况

2. 睡眠锁:

- 睡眠锁是一种基于阻塞和唤醒的同步机制，即当线程尝试获取锁时，如果锁已被其他线程持有，则该线程会进入**睡眠状态，让出CPU**，直到锁被释放并且线程被唤醒才能继续执行。
- 适用于——临界区较长且锁的争用较频繁的情况

buffer cache

文件系统的实现

文件描述符 (File descriptor)
路径名 (Pathname)
目录 (Directory)
索引结点 (Inode)
日志 (Logging)
缓冲区高速缓存 (Buffer cache)
磁盘 (Disk)

磁盘层：在virtio磁盘上读写块

Buffer cache：缓存磁盘块

日志层：允许上层通过事务更新多个磁盘块

阅读bio.c代码

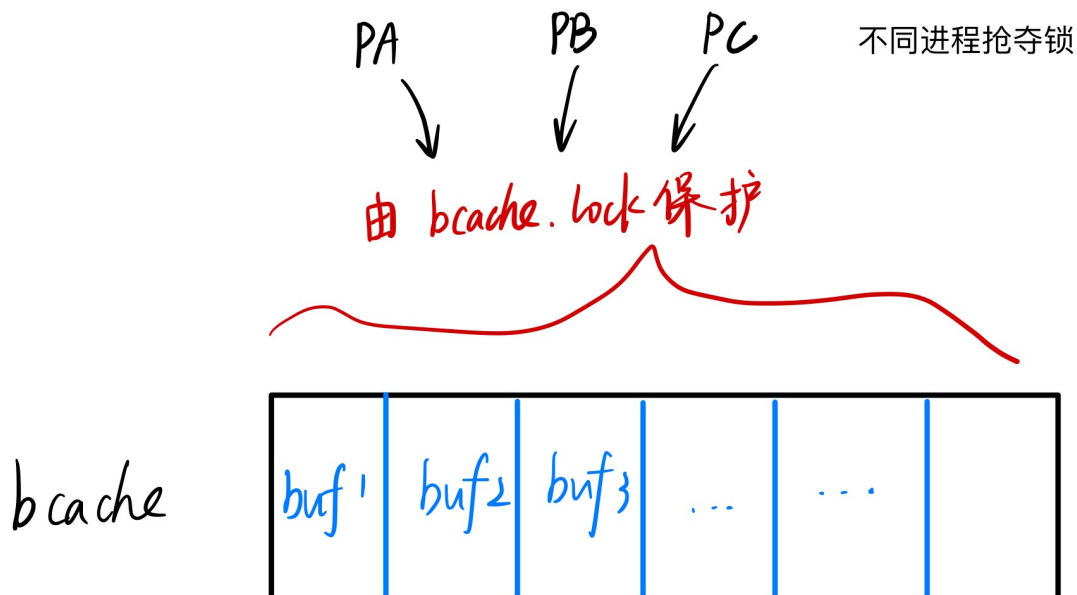
//缓冲区 (buffer) 的数据结构, 用于在内存中缓存从磁盘读取的数据。

```
struct buf {
    int valid;
    int disk;
    uint dev;//设备号
    uint blockno;//block number
    struct sleeplock lock;
    uint refcnt;//记录有多少进程在使用该缓冲块
    struct buf *prev;
    struct buf *next;
    uchar data[BSIZE];
};
```

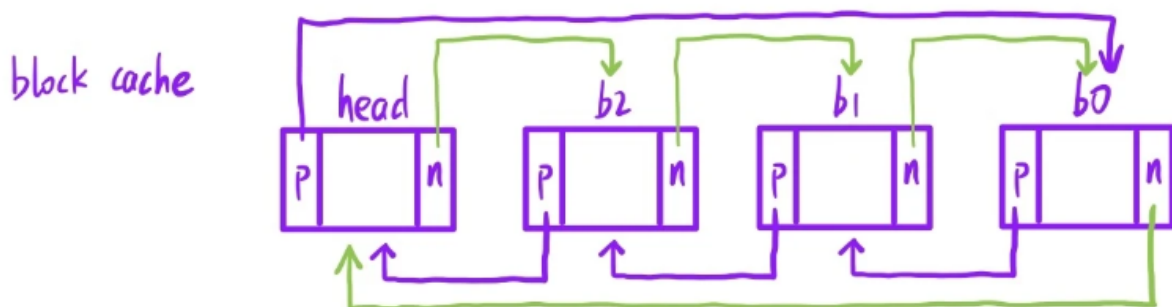
```
struct {
    struct spinlock lock;
    struct buf buf[NBUF]; //这是一个缓冲区数组, 共30个缓冲块供上层使用

    // Linked list of all buffers, through prev/next.
    // Sorted by how recently the buffer was used.
    // head.next is most recent, head.prev is least.
    struct buf head; //这是一个双向链表的头节点
} bcache;
```

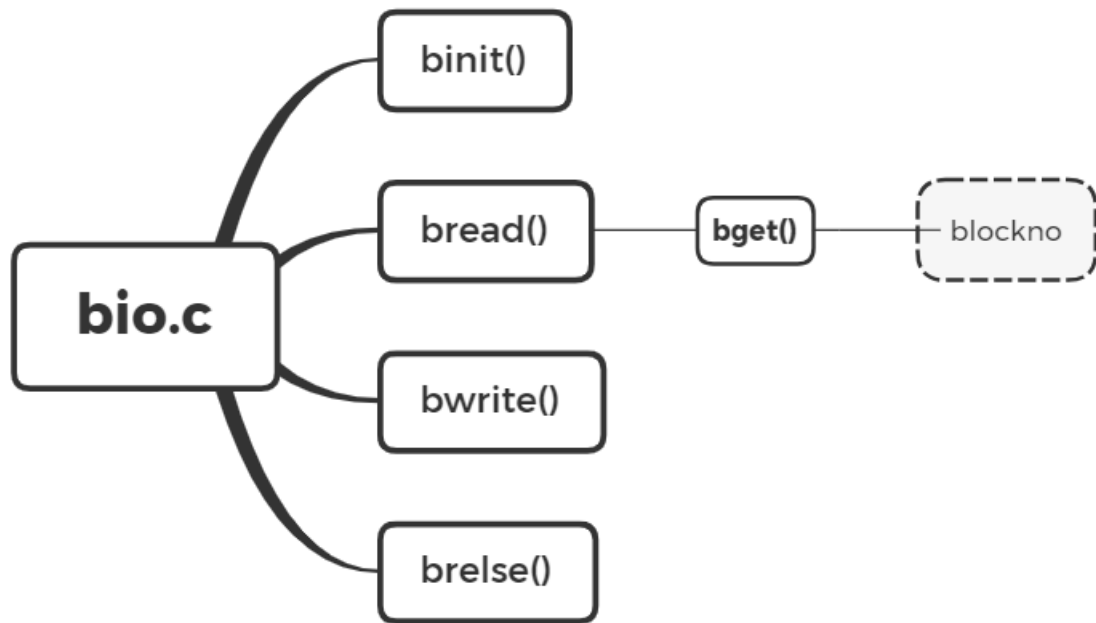
原始buffer cache



- 存储形式: 双向链表 (最近使用的buf放在head)



函数结构



利用Hash思想改进bcache锁

